

TP3 : Apprentissage supervisé et intelligence artificielle explicable

02 Juin 2026

Dans ce troisième TP, nous revenons sur l'apprentissage supervisé et nous étudions l'Intelligence Artificielle explicable. Le langage de programmation utilisé est toujours **Python**. Vous utiliserez toujours l'environnement virtuel *TP_VD* du TP1. Pour toutes les manipulations, n'hésitez pas à consulter la documentation des méthodes et bibliothèques concernées. L'utilisation d'un notebook est toujours fortement encouragée.

Apprentissage supervisé : classification

Dans cette partie, nous revoyons l'apprentissage supervisé. L'objectif est de découvrir l'apprentissage supervisé avec la bibliothèque scikit-learn. La tâche d'apprentissage supervisé qui nous intéresse ici est la classification.

1. En utilisant la fonction *load_breast_cancer* de *sklearn.datasets* (https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html), charger le jeu de données Breast Cancer Wisconsin. Les attributs de ce jeu de données sont extraits de l'analyse d'IRM de patients. Stockez les x_i dans une matrice X et les étiquettes (classes) dans un vecteur y . En analysant X et y , combien de classes y a-t-il ? Combien de données (instances) contient votre jeu de données ? Combien de dimensions (attributs) ?

Dans le jeu de données tel que fournit par *sklearn.datasets*, l'étiquette "tumeur maligne" est à 0 et l'étiquette "tumeur bénigne" est à 1. C'est contre-intuitif car 0 représente "l'absence". Dans le vecteur y , changez les 1 en 0 et les 0 en 1 pour obtenir des étiquettes plus intuitives.

2. En apprentissage supervisé, il est recommandé d'utiliser une partie du jeu de données étiquetté pour **entraîner** le modèle, et le reste du jeu de données étiquetté pour **tester** le modèle entraîné. Lorsque la performance sur le jeu de données de test est acceptable, le modèle est alors déployé en production et peut être utilisé sur pour prédire la classe de nouvelles instances dont on ne connaît pas l'étiquette.

Séparer le jeu de données en un **jeu d'entraînement** et un **jeu de test** en utilisant la fonction *train_test_split* de *sklearn.model_selection* (https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html). Utilisez 70% du jeu de données pour l'entraînement et les 30% restants pour les tests. En utilisant la fonction *train_test_split*, fixez le random state à 42. Le jeu de données d'entraînement sera contenu dans les variables X_{train} et y_{train} , et le jeu de données de test dans les variables X_{test} et y_{test} .

3. Normaliser le jeu de données en utilisant le *Standard Scaler* de *sklearn.preprocessing*

- (<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>). *N. B. : Ce sont les attributs qui doivent être normalisés. Pas besoin de normaliser y qui ne contient que les étiquettes.* Vous devez normaliser *X_train* et *X_test* séparément.
4. Entraîner un classifieur SVM en utilisant *y_train* et la version normalisée de *X_train*. Pour cela, importez et utilisez la classe *SVC* de *sklearn.svm* (<https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>). Utilisez une fonction de noyau (kernel) **rbf** et mettez le paramètre *probability* à *True* (nous servira au moment de la génération d'explications). Lorsque ce paramètre vaut *True*, le modèle peut renvoyer la probabilité d'appartenance à chaque classe à l'étape d'inférence. Lorsqu'il vaut *False*, seul l'index de la classe prédite est renvoyé pendant l'inférence. Utilisez la méthode *fit* de l'instance de *SVC* créée pour entraîner votre modèle sur *y_train* et la version normalisée de *X_train*.
 5. Calculer les prédictions de chaque instance de notre jeu de données de test. Utilisez la fonction *predict* de l'instance de *SVC* créée pour obtenir les prédictions de chaque instance dans la version normalisée de *X_test*. Stockez ces prédictions dans un vecteur *y_pred*. *y_pred* vous donne ainsi les prédictions de notre classifieur, tandis que *y_test* contient les vraies étiquettes.
 6. Pour évaluer notre classifieur, nous allons calculer sa précision et son rappel. La classe "tumeur maligne" est définie comme classe positive et la classe "tumeur bénigne" est considérée comme classe négative. La **précision** nous indique quelle proportion de patients prédits comme faisant partie de la classe positive possédait réellement une tumeur maligne. Le **rappel** nous indique quelle proportion de patients possédant réellement une tumeur maligne ont été identifiés par notre classifieur. Utilisez les fonctions *precision_score* et *recall_score* de *sklearn.metrics* pour calculer la précision et le rappel de notre classifieur.

LIME

Dans cette partie, nous utilisons LIME pour générer des explications de notre classifieur. La documentation de LIME se trouve ici : <https://lime-ml.readthedocs.io/en/latest/lime.html#>.

1. Installer LIME dans votre environnement *TP_VD* avec *pip*.
2. Instancier un objet **explainer** LIME de la classe *LimeTabularExplainer* de *lime.lime_tabular* (https://lime-ml.readthedocs.io/en/latest/lime.html#module-lime.lime_tabular). Certains paramètres doivent être fixés.
3. Quelle est la classe prédite par notre modèle sur l'instance d'indice **10** du jeu de données de test ? Expliquer cette instance (utiliser le jeu de données normalisé bien sûr, car nous avons construit et validé le modèle sur les ensembles normalisés). Pour ce faire, utilisez la fonction *explain_instance* de votre explainer de la question **2** (https://lime-ml.readthedocs.io/en/latest/lime.html#lime.lime_tabular.LimeTabularExplainer.explain_instance). Veillez à obtenir une explication pour **la classe prédite** (paramètre

labels). Cette fonction vous renvoie un objet de la classe *Explanation* (<https://lime-ml.readthedocs.io/en/latest/lime.html#lime.explanation.Explanation>).

4. Visualiser l'explication précédente en utilisant les fonctions *show_in_notebook* (préférable) ou *as_pyplot_figure* de l'objet retourné à la question 3.
5. Que traduit l'explication ? Quelles sont les attributs les plus importants ?
6. Faire varier le nombre d'attributs de l'explication (paramètre *num_features* de la fonction *explain_instance*) et observer les résultats.
7. Expliquez d'autres instances de votre choix. N. B. : pas besoin de réinstancier l'objet *explainer*.

SHAP

Dans cette partie, nous utilisons SHAP pour générer des explications de notre classifieur. La documentation de SHAP se trouve ici : <https://shap.readthedocs.io/en/latest/>.

1. Installez SHAP dans votre environnement *TP_VD* avec *pip*.
2. Instancier un objet **explainer** SHAP de la classe *explainers.Sampling* de *shap* (<https://shap.readthedocs.io/en/latest/generated/shap.SamplingExplainer.html#shap.SamplingExplainer>).
3. Calculer les valeurs SHAP de l'instance d'indice 10 du jeu de données de test (normalisé). Ce sont des approximations des valeurs de Shapley. Pour ce faire, utilisez la fonction *shap_values* de votre *explainer* de la question 2.
4. Afficher les valeurs shap calculées. Essayez de les interpréter.
5. Visualiser l'explication précédente en utilisant la fonction *decision* de *shap.plots* (<https://shap.readthedocs.io/en/latest/generated/shap.plots.decision.html>). Cette fonction requiert comme premier argument la **prédiction moyenne** de la classe considérée. Elle est contenue dans l'attribut *expected_value* de votre *explainer* de la question 2. Veillez à visualiser l'explication pour la **classe prédite** uniquement.
6. Que traduit l'explication ? Quels sont les attributs les plus importants ?
7. Comparer l'explication de SHAP et celle de LIME.
8. Visualiser l'explication de la question 4 en utilisant la fonction *force* de *shap.plots*. Observez la différence entre le graphique obtenu et celui de la question 5. Quelle(s) information(s) supplémentaire(s) ce graphique vous apporte t-il ?
9. Expliquez d'autres instances de votre choix.